



`std::to_signed` and `std::to_unsigned`

Document number:

[P3643R2](#)

Date:

2025-09-29

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3643/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/to-signed-unsigned.cow



ABSTRACT: Add `std::to_signed` and `std::to_unsigned` function templates in the style of `std::to_underlying`.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R1
 - 1.2 Changes since R0
- 2 **Introduction**
 - 2.1 Existing developer practice
 - 2.2 Dual purpose
- 3 **Impact on the standard**
- 4 **Design**
 - 4.1 Constraints
- 5 **Possible implementation**

6	Wording
7	Acknowledgements
8	References



§ 1. Revision history

§ 1.1. Changes since R1

- ported proposal from Bikeshed to COWEL
- added §2.2. Dual purpose
- reordered §4. Design and §5. Possible implementation
- added discussion on constraints in §4. Design
- fixed typo in §6. Wording: ~~x~~ value

§ 1.2. Changes since R0

- tightened constraints in §6. Wording to exclude cv-qualified types in general, not just cv `bool`
- added *freestanding* comment to feature-test macro

§ 2. Introduction

In integer numerics and bit-manipulation code, it is common to implement functionality in terms of the corresponding signed/unsigned type:

```
template<class T>
T arithmetic_shift_right(T x, int s) {
    return T(std::make_signed_t<T>(x) >> s);
}

template<class T>
T wrapping_add(T x, T y) {
    constexpr unsigned to_int_promotion_defense = 0;
    return T(to_int_promotion_defense
        + std::make_unsigned_t<T>(x)
        + std::make_unsigned_t<T>(y));
}
```

This technique *technically* works, but suffers from some problems:



- The use of C-style/function-style casts may conflict with the project's style. When `static_cast` is used instead, this code becomes substantially more verbose.
- Since `T` has to be spelled out in code, abbreviated function templates cannot be used instead (or at least, `decltype(x)` would be required for the cast).
- Repeating the type `T` may be prone to mistakes. Nothing guarantees us that `T` is of type `T` when writing an expression `std::make_signed_t<T>(x)`. In larger code samples, mismatching types and variables is a bug waiting to happen. To be safe, we would have to write `std::make_signed_t<decltype(x)>(x)`. However, this still repeats the expression `x`.

The greater the distance between `T` and the use of `std::make_{un}signed` are, the easier it is to make a mistake.

To solve these issues, this proposal adds the function templates `std::to_signed(x)` and `std::to_unsigned(x)`, which deduce `T` from `x`. This is concise and always uses the correct type.

§ 2.1. Existing developer practice

A GitHub code search for

```
" "[/^a-zA-Z_](to_signed|to_unsigned)|static_cast<(typename)? ?((::)?std::)?
(make_signed|make_unsigned)/
-is:fork language:c++
```

... shows that roughly 13.8K C++ files already use a non-standard `to_unsigned` and `to_signed`, or which `static_cast` to `make_signed` or `make_unsigned`.

By comparison,

```
" "[/^a-zA-Z_](to_underlying)|static_cast<(typename)? ?((::)?std::)?(underlying_type)/
-is:fork language:c++
```

... yields 43K C++ files which use `to_underlying`, or which `static_cast` to `std::underlying_type`.

The proposal [\[P1682R3\]](#) for `std::to_underlying` had similar rationale, and at the time, the author was only able to discover 1000 search results for `to_underlying`.

§ 2.2. Dual purpose

In addition to the purpose of changing signedness of integers, the proposed functions have another purpose: mapping other integral and enumeration types onto signed or unsigned integer types.



EXAMPLE: Assuming 32-bit `unsigned`, the use of `to_unsigned` reduces the amount of template instantiations below:



```
// BAD, two instantiations of popcnt that are functionally equivalent:
popcnt(U'x');           // calls popcnt<char32_t>
popcnt(123u);           // calls popcnt<unsigned>

// GOOD, only one instantiation of popcnt:
popcnt(to_unsigned(U'x')); // calls popcnt<unsigned>
popcnt(to_unsigned(123u)); // calls popcnt<unsigned>
```

Of course, the instantiation of `to_unsigned` is added, but its cost is quickly outweighed by avoiding instantiations of much larger and complicated templates throughout the code base.

Just like `make_signed` and `make_unsigned`, `to_signed` and `to_unsigned` map the much larger set of integral types onto the smaller set of signed and unsigned integer types. This reduces template instantiations and may improve compilation speed and reduce code size.

§ 3. Impact on the standard

This proposal is a pure library extension.

§ 4. Design

This proposal follows precedent: Similar to `to_underlying`, the proposed functions are located in `<utility>`. The naming scheme is based on `to_underlying` and the search results in §2.1. Existing developer practice.

§ 4.1. Constraints

The design of `to_signed` and `to_unsigned` should be SFINAE-friendly because that is useful.



EXAMPLE: Being able to test whether `std::to_signed(x)` is valid lets us constrain the interface of `arithmetic_shift_right`.

```
template<class T>
    requires requires (T x) { std::to_signed(x); }
T arithmetic_shift_right(T x, int s) {
    return T(std::to_signed(x) >> s);
}
```

As for the constraints of the proposed functions, there are multiple options:

- **✗ Only permit signed or unsigned integer types.** This would be needlessly restrictive because e.g. `make_signed<char16_t>` is also valid (and yields e.g. `signed short`), which makes it possible to implement the `arithmetic_shift_right` function in §2. [Introduction](#) for that `char16_t` as well. In general, it would make generic programming harder if only some of the integral types were supported.
- **✓ Permit what `make_signed` supports.** That is, integral or enumeration types other than `bool`. This is proposed. There is no compelling reason from deviating from the restrictions imposed by `make_signed` and `make_unsigned`, as their restrictions de-facto define for which types a sign conversion makes sense.
- **✗ Permit *integer-like* types.** This is out-of scope, and would effectively replace *to-unsigned-like*. Integer-like types are an open set of types, possibly including user-defined types. This effectively turns the proposed function into a customization point. Seeing that *integer-like* is a concept which only exists in [\[ranges\]](#), the proposed functions should then be `std::ranges::to_signed` and `std::ranges::to_unsigned`. However, the goal of the proposal is to provide a simple numerical function, not add a ranges-specific customization point object.
- **✗ Permit arithmetic types.** This would mirror `std::is_signed_v`, which is also `true` for floating-point types. This would create a weird asymmetry, where `to_signed(float)` is a no-op, and `to_unsigned(float)` is ill-formed. The result of the proposed functions is always a signed or unsigned integer type, and permitting `float` would muddy the waters here.

§ 5. Possible implementation

```
template<class T>
concept __sign_convertible
    = (is_integral_v<T> || is_enum_v<T>)
    && !is_same_v<T, bool>
    && !is_const_v<T>
    && !is_volatile_v<T>;

template<__sign_convertible T>
constexpr make_signed_t<T> to_signed(T x) noexcept {
    return static_cast<make_signed_t<T>>(x);
}

template<__sign_convertible T>
constexpr make_unsigned_t<T> to_unsigned(T x) noexcept {
    return static_cast<make_unsigned_t<T>>(x);
}
```

§ 6. Wording

The proposed wording is relative to [\[N5014\]](#).

In subclause [\[version.syn\]](#), add the following feature-testing macro:



```
#define __cpp_lib_to_signed 20XXXXL // freestanding, also in <utility>
```

In subclause [\[utility.syn\]](#), change the synopsis as follows:



```
namespace std {  
    [...]  
  
    // [utility.sign.conv], sign conversion  
    template<class T>  
        constexpr make_signed_t<T> to_signed(T value) noexcept;  
    template<class T>  
        constexpr make_unsigned_t<T> to_unsigned(T value) noexcept;  
  
    // [utility.underlying], to_underlying  
    template<class T>  
        constexpr underlying_type_t<T> to_underlying(T value) noexcept;  
  
    [...]  
}
```

In subclause [\[utility\]](#), add a subclause immediately prior to [\[utility.underlying\]](#):



Sign conversion

[\[utility.sign.conv\]](#)

```
template<class T>  
    constexpr make_signed_t<T> to_signed(T value) noexcept;  
  
    Constraints: T is a cv-unqualified integral or enumeration type other than bool.  
  
    Returns: static_cast<make_signed_t<T>>(value) ([meta.trans.sign]).  
  
template<class T>  
    constexpr make_unsigned_t<T> to_unsigned(T value) noexcept;  
  
    Constraints: T is a cv-unqualified integral or enumeration type other than bool.  
  
    Returns: static_cast<make_unsigned_t<T>>(value) ([meta.trans.sign]).
```



WARNING: Because `make_unsigned_t` uses *Mandates*, `to_unsigned` is arguably not SFINAE-friendly. For example, `make_unsigned_t<int*>` is ill-formed (?) without being instantiated. I don't attempt to fix this; I assume that the above specification "just works".

In practice, compilers do not eagerly instantiate `make_signed_t` when the constraints of the function template are not satisfied, so this specification works.

§ 7. Acknowledgements



Thanks to Dalton Messmer for reviewing R1 of this paper, leaving thoughtful comments that contributed to an improved R2 which discusses many design aspects in more detail.

§ 8. References

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P1682R3] *JeanHeyd Meneide*. `std::to_underlying` for enumerations (2021-01-22)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1682r3.html>